

Master Checklist

Frontend · Java Backend · Python Backend · AI Engineer — check off as you go

[CORE] = needed for fresher-level resumes/interviews now [STRETCH] = differentiator depth, fine to defer past placements

★ Updated for Shard / Cairn (distributed cache twins): Caching, Database Sharding, and Consistent Hashing moved from HLD-STRETCH to CORE since they're now hands-on build items, not just theory. Added explicit ReadWriteLock / ScheduledExecutorService, Spring Boot Actuator, and Strategy Pattern call-outs where the two projects exercise them directly.

Phase 0: Setup / Foundations [CORE]

- ☐ Linux basics (cd, ls, cat, grep)
- ☐ Git + GitHub (branches, merge conflicts, revert vs reset, feature→main flow)
- ☐ .env / secrets discipline
- ☐ Debugging basics
- ☐ Reading stack traces
- ☐ IDE + Browser DevTools

Phase 1: Programming Basics [CORE]

- ☐ Java syntax
- ☐ Variables + Data Types
- ☐ Conditions + Loops
- ☐ Functions/Methods
- ☐ Arrays + Strings
- ☐ Input/Output

Phase 2: OOP [CORE]

- ☐ Classes & Objects
- ☐ Encapsulation
- ☐ Inheritance
- ☐ Polymorphism
- ☐ Abstraction
- ☐ Interfaces
- ☐ Composition vs Inheritance
- ☐ Practice projects: ATM, Library Management, Parking Lot

Phase 3: Core Java Advanced [CORE]

- ☐ Collections Framework (List/Set/Queue/Map, ArrayList vs LinkedList, HashMap vs TreeMap vs LinkedHashMap, HashSet vs TreeSet)
- ☐ Exception Handling (checked vs unchecked, try/catch/finally gotcha, custom exceptions)
- ☐ File Handling
- ☐ Generics (type safety, wildcards, PECS)
- ☐ == vs .equals(), pass-by-value semantics

- ☐ static vs instance members
- ☐ Java 8+ Features
- ☐ Lambdas & functional interfaces
- ☐ Streams API (intermediate vs terminal ops, filter/map/flatMap/collect/reduce)
- ☐ Optional
- ☐ Multithreading (Process vs Thread, Thread vs Runnable vs Callable)
- ☐ Synchronization (synchronized, volatile, ReentrantLock)
- ★ Read-write locks (ReadWriteLock) and scheduled background tasks (ScheduledExecutorService) — practiced directly in Cairn's cache-engine concurrency model
- ☐ Atomic variables
- ☐ ExecutorService & thread pools (incl. RejectedExecutionException when the queue fills up)
- ☐ CompletableFuture
- ☐ Concurrent Collections
- ☐ Memory model (Stack vs Heap, String Pool, string immutability, intern())
- ☐ Garbage Collection (Mark-Sweep-Compact; Eden, Survivor S0/S1, Old/Tenured Generation, Metaspace)
- ☐ Enums, Comparable vs Comparator

Phase 4: DSA [CORE — parallel track, never pauses]

Full pattern-based breakdown lives in DSA_Master_Checklist.docx (Phase 1: Arrays/Strings, Stacks/Queues, Linked Lists, Hash Maps/Sets, Trees, Graphs, Tries, JCF; Phase 2: all 16 patterns with sample problems).

- ☐ Practice: LeetCode, NeetCode, Striver Sheet
- ☐ Discipline: one pattern for 5-7 days, brute force before optimizing, explain approach out loud

Phase 5: Database [CORE]

- ☐ SQL Basics (SELECT, WHERE, GROUP BY, ORDER BY)
- ☐ Joins (INNER vs LEFT)
- ☐ Primary Keys vs Foreign Keys
- ☐ Indexes
- ☐ Query Optimization
- ☐ Transactions
- ☐ ACID
- ☐ Normalization & relationships (one-to-many, many-to-many)
- ☐ Connection pooling
- ☐ Relational vs non-relational (pgvector/FAISS use case)
- ☐ PostgreSQL
- ☐ MySQL
- ☐ Redis
- ☐ psql / DBeaver / pgAdmin

Phase 6: Backend Development [CORE (pick your resume tracks below)]

- ☐ HTTP + REST APIs (verbs, idempotency, status codes, 401 vs 403)

- ☐ JSON
- ☐ Statelessness (REST & JWT)
- ☐ Maven/Gradle
- ☐ Swagger/OpenAPI Documentation

Spring Boot (Java Backend / Full Stack resume) [CORE]

- ☐ Spring Core
- ☐ Dependency Injection
- ☐ @RestControllerAdvice
- ☐ REST APIs
- ☐ Validation
- ☐ Exception Handling
- ☐ JDBC (raw Connection/Statement/PreparedStatement/ResultSet, before the ORM abstracts it away)
- ☐ Spring Data JPA + Hibernate
- ☐ Entity Relationships
- ☐ Pagination
- ☐ Spring Security
- ☐ JWT Authentication
- ☐ Role-Based Auth
- ☐ OAuth app registration
- ☐ Microservices
- ☐ API key provisioning & quota/cost management across multiple providers
- ☐ Spring AI ChatClient (see Phase 11)
- ★ Spring Boot Actuator — metrics/health endpoints, used for Cairn's Phase 3 hit/miss ratio, eviction count, and latency percentile tracking

Django (Python Backend / Full Stack resume) [CORE]

- ☐ Django models, migrations, DRF serializers/viewsets
- ☐ Django Channels + ASGI
- ☐ Daphne / Uvicorn vs WSGI
- ☐ Celery + Redis (background job processing)
- ☐ Object-level permissions
- ☐ File upload handling
- ☐ Search/filter query design (tag-based)
- ☐ State machine design (status transitions)
- ☐ Flyway migrations (Java-side equivalent)

Python / FastAPI (Python Backend / AI Engineer resume) [CORE]

- ☐ Python fundamentals (for AI/backend use)
- ☐ FastAPI

Testing [CORE]

- ☐ JUnit
- ☐ Mockito
- ☐ Integration Testing

- ☐ Pytest (deterministic pass/fail harness)

Phase 7: Frontend [CORE (Frontend resume)]

- ☐ HTML, CSS, JavaScript
- ☐ DOM, Async JS, Fetch API
- ☐ React (Components, Props + State, Hooks: useState/useEffect/useRef)
- ☐ Routing
- ☐ Forms
- ☐ API Integration
- ☐ Context API
- ☐ State Management
- ☐ TanStack Query
- ☐ Tailwind CSS
- ☐ TypeScript (types, interfaces)
- ☐ Vite
- ☐ Vercel deployment
- ☐ Web Workers
- ☐ WebAssembly (conceptual)

Phase 8: DevOps + Deployment [CORE (baseline) / STRETCH (monitoring)]

- ☐ Docker
- ☐ Docker Compose (multi-container networking)
- ☐ Docker SDK for Python
- ☐ Container resource limits
- ☐ CI/CD Basics
- ☐ GitHub Actions
- ☐ Nginx Basics

AWS [CORE]

- ☐ EC2
- ☐ S3
- ☐ RDS
- ☐ ECS + Fargate
- ☐ ElastiCache
- ☐ ECR
- ☐ Deploy Spring Boot
- ☐ Deploy React App
- ☐ HTTPS + Domain

Monitoring & Logging [STRETCH]

- ☐ Application Logs
- ☐ Monitoring Basics
- ☐ Prometheus (optional)

- ☐ Grafana (optional)

Phase 9: LLD [STRETCH — some product companies ask fresher-level LLD]

- ☐ SOLID Principles
 - ★ Design Patterns, incl. Adapter Pattern and Strategy Pattern (Shard/Cairn's pluggable LRU/LFU eviction policies are a direct, nameable Strategy-pattern implementation)
- ☐ UML Basics
- ☐ Object Modeling
- ☐ Design problems: Parking Lot, BookMyShow, Splitwise, Elevator System, Chess

Phase 10: HLD

Caching, Database Sharding, and Consistent Hashing moved to CORE below — Shard/Cairn make these hands-on build items rather than pure theory. The rest stays STRETCH (rarely gates a fresher screen).

CORE (practiced via Shard/Cairn)

- ★ Caching — custom in-memory cache engine, LRU/LFU eviction, TTL expiry
- ★ Database Sharding — static sharding across cache nodes
- ★ Consistent Hashing — routing keys to nodes, rebalancing on node add/remove

STRETCH

- ☐ Scalability Basics
- ☐ Load Balancer
- ☐ Kafka/RabbitMQ
- ☐ API Gateway
- ☐ Microservices
- ☐ Rate Limiting
- ☐ CDN
- ☐ CAP Theorem
- ☐ Sticky session routing (WebSocket scaling)
- ☐ Design problems: YouTube, WhatsApp, Uber, Netflix

Phase 11: GenAI + AI Integration [CORE (AI Engineer resume)]

AI Basics

- ☐ Neural Networks / Deep Learning basics
- ☐ LLM Fundamentals
- ☐ How LLMs Work internally
- ☐ Prompt Engineering
- ☐ Embeddings
- ☐ Vector Databases (pgvector, FAISS)
- ☐ Chunking
- ☐ Hybrid search (vector + keyword)
- ☐ Confidence scoring
- ☐ RAG (Retrieval-Augmented Generation)

- ☐ Function/Tool Calling
- ☐ AI Agents Basics
- ☐ LangGraph (agent orchestration)
- ☐ Decision log / agent memory pattern
- ☐ Multi-provider integration (OpenAI, Gemini, Claude/Anthropic APIs)
- ☐ LLMOps

Spring AI [STRETCH — depth beyond fresher AI Engineer bar]

- ☐ Spring AI Basics
- ☐ ChatClient
- ☐ Prompt Templates
- ☐ AI Model Providers
- ☐ OpenAI Integration
- ☐ Gemini Integration
- ☐ Ollama Integration
- ☐ Embeddings in Spring AI
- ☐ VectorStore Integration
- ☐ RAG using Spring AI
- ☐ Tool Calling in Spring AI
- ☐ AI Memory
- ☐ AI Agents with Spring AI
- ☐ MCP (Model Context Protocol) Basics
- ☐ AI Streaming Responses

Phase 12: QA / Specialized Testing [CORE (QA resume)]

- ☐ Playwright (locators, assertions, headless, Page Object Model)
- ☐ API assertions
- ☐ k6 (virtual users, ramping stages, load vs stress testing, p95/p99 latency)
- ☐ Grafana output interpretation
- ☐ OWASP Top 10 (SQLi, XSS, broken access control)
- ☐ OWASP ZAP
- ☐ Custom access-control (IDOR-style) testing
- ☐ Input fuzzing
- ☐ Anomaly detection (rule-based)

Phase 13: Build Projects [CORE]

- ☐ Build 2-3 high-quality Full Stack + AI-Integrated projects
- ☐ Sandbox lifecycle debugging (provision → execute → monitor → tear down)
- ☐ Docker networking debugging
- ☐ ASGI misconfiguration debugging
- ☐ WASM load failure debugging
- ☐ Stuck worker / orphaned job debugging
- ☐ Container cleanup guarantees on crash

Phase 14: Interview Topics [CORE]

Java

- ☐ JVM
- ☐ Garbage Collection
- ☐ HashMap Internals
- ☐ Multithreading
- ☐ Synchronization
- ☐ Immutable Classes

Spring Boot

- ☐ Bean Lifecycle
- ☐ Dependency Injection
- ☐ Security
- ☐ Transactions
- ☐ Hibernate Internals
- ☐ Lazy vs Eager Loading

Database

- ☐ Indexing
- ☐ Query Optimization
- ☐ ACID
- ☐ Locking
- ☐ Joins
- ☐ Pagination

System Design [STRETCH]

- ☐ Scaling
- ☐ Caching
- ☐ DB Design
- ☐ Rate Limiting
- ☐ Messaging Queues
- ☐ Consistency vs Availability